LAB 05

Stage v (Verify): Home Activities

Defining Data, Strings and Arrays in Assembly Language (MASM)

Submitted by	Muhammad Ahsan
Registration No.	FA24-BCS-059
Submitted to	Sir Taimur Shahzad
Subject	COAL
Lab	Lab 05: Stage v (Verify), Home Activities

Contents and Setup

Each home task is a complete 32-bit MASM program that uses Kip Irvine's **Irvine32** library (the setup used in the lab with Visual Studio). Every program declares the data the task asks for and then prints it, so the output can be checked against the declarations.

Task	Topic	Program
Home Task 1	Defining data (BYTE, WORD, DWORD, SBYTE, ?)	`HomeTask1.asm`
Home Task 2	A multi-line menu declared as one string	`HomeTask2.asm`
Home Task 3	1-D, 2-D and 3-D arrays with DUP	`HomeTask3.asm`
Home Task 4	C++ char / int declarations in MASM	`HomeTask4.asm`

How the programs were built and run

- Assembler: JWasm v2.21, a MASM-compatible assembler that accepts the same source as Visual Studio's `ml.exe`.
- Linker: JWlink, linking each program with `Irvine32.lib`, `Kernel32.lib` and `User32.lib` into a 32-bit Windows console program (`.exe`).
- Run: each `.exe` was started from `cmd.exe` in a Win32 console window (using Wine), and the screenshots show that window.
- The same `.asm` files build in Visual Studio with the usual Irvine32 project: add the file to the project and press Ctrl+F5.

Home Task 1: Defining Data

Task statement

Define the following data in Assembly:

```
Sort   BYTE  'y'          ; ASCII of y = 79H
value  WORD  25159        ; 25159D = 6247H
total  DWORD 542803535    ; 542803535D = 205A864FH
marks  DWORD 0, 0, 0, 0, 0, 0, 0, 0
value1 BYTE  'A'          ; character constant
value2 BYTE  0            ; smallest unsigned byte
value3 BYTE  255          ; largest unsigned byte
value4 SBYTE -128         ; smallest signed byte
value5 SBYTE +127         ; largest signed byte
value6 BYTE  ?            ; uninitialized byte
```

Declarations (answer)

Variable	Type	Initializer	Hex value	Bytes in memory
Sort	BYTE	'y'	79h	79
value	WORD	25159	6247h	47 62
total	DWORD	542803535	205A864Fh	4F 86 5A 20
marks	DWORD	0, 0, 0, 0, 0, 0, 0, 0	8 x 00000000h	32 x 00
value1	BYTE	'A'	41h	41
value2	BYTE	0	00h	00
value3	BYTE	255	FFh	FF
value4	SBYTE	-128	80h	80
value5	SBYTE	+127	7Fh	7F
value6	BYTE	? (uninitialized)	00h	00

Source code: HomeTask1.asm

```
1  TITLE Lab 05 - Home Task 1 : Defining Data            (HomeTask1.asm)
2
3  ;-----------------------------------------------------------------------
4  ; Name         : Muhammad Ahsan
5  ; Reg. No.     : FA24-BCS-059
6  ; Subject      : COAL
7  ; Submitted to : Sir Taimur Shahzad
8  ; Lab 05       : Stage v (verify) - Home Activities
9  ;
10 ; Home Task 1  : Define the given data in assembly language, then
11 ;                display every variable in hexadecimal and decimal
12 ;                and dump the memory that holds all of them.
13 ;-----------------------------------------------------------------------
14
15 INCLUDE Irvine32.inc
16
17 .data
18 ;------------------ Home Task 1 : data definitions ------------------
19 Sort    BYTE   'y'                    ; ASCII of y = 79H
20 value   WORD   25159                  ; 25159D = 6247H
21 total   DWORD  542803535              ; 542803535D = 205A864FH
22 marks   DWORD  0, 0, 0, 0, 0, 0, 0, 0   ; array of eight doublewords
```

```
23  value1  BYTE   'A'                     ; character constant
24  value2  BYTE   0                       ; smallest unsigned byte
25  value3  BYTE   255                     ; largest unsigned byte
26  value4  SBYTE  -128                    ; smallest signed byte
27  value5  SBYTE  +127                    ; largest signed byte
28  value6  BYTE   ?                       ; uninitialized byte
29
30  DataSize = ($ - Sort)                  ; bytes used by the data above
31
32  ;---------------------- strings for the output ----------------------
33  banner1 BYTE "Lab 05 - Home Task 1 : Defining Data in Assembly",0
34  banner2 BYTE "Muhammad Ahsan  |  FA24-BCS-059  |  COAL",0
35  dashes  BYTE "------------------------------------------------",0
36  header  BYTE "Variable  Type    Hex          Decimal",0
37  hdrLine BYTE "--------  ------  -----------  ----------------",0
38
39  lSort   BYTE "Sort      BYTE    ",0
40  lValue  BYTE "value     WORD    ",0
41  lTotal  BYTE "total     DWORD   ",0
42  lMarks  BYTE "marks     DWORD   ",0
43  lValue1 BYTE "value1    BYTE    ",0
44  lValue2 BYTE "value2    BYTE    ",0
45  lValue3 BYTE "value3    BYTE    ",0
46  lValue4 BYTE "value4    SBYTE   ",0
47  lValue5 BYTE "value5    SBYTE   ",0
48  lValue6 BYTE "value6    BYTE    ",0
49
50  sOpen   BYTE "  ('",0
51  sClose  BYTE "')",0
52  sEach   BYTE "8 elements   ",0
53  sMarks1 BYTE "LENGTHOF = ",0
54  sMarks2 BYTE ", SIZEOF = ",0
55  sMarks3 BYTE " bytes",0
56  sUninit BYTE "  (uninitialized ?)",0
57  sDump   BYTE "Memory dump of all the variables above (",0
58  sDump2  BYTE " bytes, little-endian):",0
59
60  .code
61  main PROC
62      ; ---- title ----
63      mov   edx, OFFSET banner1
64      call  WriteString
65      call  Crlf
66      mov   edx, OFFSET banner2
67      call  WriteString
68      call  Crlf
69      mov   edx, OFFSET dashes
70      call  WriteString
71      call  Crlf
72      mov   edx, OFFSET header
73      call  WriteString
74      call  Crlf
75      mov   edx, OFFSET hdrLine
76      call  WriteString
77      call  Crlf
78
79      ; ---- Sort  BYTE 'y' ----
80      mov   edx, OFFSET lSort
81      call  WriteString
82      movzx eax, Sort
83      mov   ebx, TYPE Sort
84      call  ShowHex
85      call  WriteDec                  ; 121
86      mov   edx, OFFSET sOpen
```

```
    call  WriteString
    call  WriteChar                    ; y
    mov   edx, OFFSET sClose
    call  WriteString
    call  Crlf

    ; ---- value  WORD 25159 ----
    mov   edx, OFFSET lValue
    call  WriteString
    movzx eax, value
    mov   ebx, TYPE value
    call  ShowHex
    call  WriteDec
    call  Crlf

    ; ---- total  DWORD 542803535 ----
    mov   edx, OFFSET lTotal
    call  WriteString
    mov   eax, total
    mov   ebx, TYPE total
    call  ShowHex
    call  WriteDec
    call  Crlf

    ; ---- marks  DWORD 0,0,0,0,0,0,0,0 ----
    mov   edx, OFFSET lMarks
    call  WriteString
    mov   edx, OFFSET sEach
    call  WriteString
    mov   esi, OFFSET marks
    mov   ecx, LENGTHOF marks          ; 8 elements
L1: mov   eax, [esi]
    call  WriteDec
    mov   al, ' '
    call  WriteChar
    add   esi, TYPE marks
    loop  L1
    call  Crlf

    ; LENGTHOF and SIZEOF of marks
    mov   ecx, 31                      ; indent to the Decimal column
L2: mov   al, ' '
    call  WriteChar
    loop  L2
    mov   edx, OFFSET sMarks1
    call  WriteString
    mov   eax, LENGTHOF marks
    call  WriteDec
    mov   edx, OFFSET sMarks2
    call  WriteString
    mov   eax, SIZEOF marks
    call  WriteDec
    mov   edx, OFFSET sMarks3
    call  WriteString
    call  Crlf

    ; ---- value1  BYTE 'A' ----
    mov   edx, OFFSET lValue1
    call  WriteString
    movzx eax, value1
    mov   ebx, TYPE value1
    call  ShowHex
    call  WriteDec                     ; 65
    mov   edx, OFFSET sOpen
```

```
151     call  WriteString
152     call  WriteChar                     ; A
153     mov   edx, OFFSET sClose
154     call  WriteString
155     call  Crlf
156
157     ; ---- value2  BYTE 0 ----
158     mov   edx, OFFSET lValue2
159     call  WriteString
160     movzx eax, value2
161     mov   ebx, TYPE value2
162     call  ShowHex
163     call  WriteDec
164     call  Crlf
165
166     ; ---- value3  BYTE 255 ----
167     mov   edx, OFFSET lValue3
168     call  WriteString
169     movzx eax, value3
170     mov   ebx, TYPE value3
171     call  ShowHex
172     call  WriteDec
173     call  Crlf
174
175     ; ---- value4  SBYTE -128 (signed, so use MOVSX and WriteInt) ----
176     mov   edx, OFFSET lValue4
177     call  WriteString
178     movsx eax, value4
179     mov   ebx, TYPE value4
180     call  ShowHex
181     call  WriteInt
182     call  Crlf
183
184     ; ---- value5  SBYTE +127 ----
185     mov   edx, OFFSET lValue5
186     call  WriteString
187     movsx eax, value5
188     mov   ebx, TYPE value5
189     call  ShowHex
190     call  WriteInt
191     call  Crlf
192
193     ; ---- value6  BYTE ? ----
194     mov   edx, OFFSET lValue6
195     call  WriteString
196     movzx eax, value6
197     mov   ebx, TYPE value6
198     call  ShowHex
199     call  WriteDec
200     mov   edx, OFFSET sUninit
201     call  WriteString
202     call  Crlf
203
204     ; ---- memory dump of every byte defined above ----
205     call  Crlf
206     mov   edx, OFFSET sDump
207     call  WriteString
208     mov   eax, DataSize
209     call  WriteDec
210     mov   edx, OFFSET sDump2
211     call  WriteString
212     call  Crlf
213     mov   esi, OFFSET Sort              ; starting address
214     mov   ecx, DataSize                 ; number of units
```

```
215     mov   ebx, TYPE BYTE               ; unit size = 1 byte
216     call  DumpMem
217
218     exit
219 main ENDP
220
221 ;-----------------------------------------------------------------
222 ShowHex PROC
223 ;
224 ; Writes EAX in hexadecimal using EBX bytes (1, 2 or 4), adds the
225 ; suffix 'h' and pads with spaces so that the next column always
226 ; starts at the same position.
227 ; Receives: EAX = value, EBX = size of the value in bytes
228 ; Returns:  nothing (EAX, EBX and ECX are preserved)
229 ;-----------------------------------------------------------------
230     push  eax
231     push  ecx
232     call  WriteHexB                    ; 2, 4 or 8 hex digits
233     mov   al, 'h'
234     call  WriteChar
235     mov   ecx, 12                      ; spaces = 12 - 2 * size
236     sub   ecx, ebx
237     sub   ecx, ebx
238     mov   al, ' '
239 SH1:call  WriteChar
240     loop  SH1
241     pop   ecx
242     pop   eax
243     ret
244 ShowHex ENDP
245
246 END main
```

Output

Console window after running `HomeTask1.exe`:

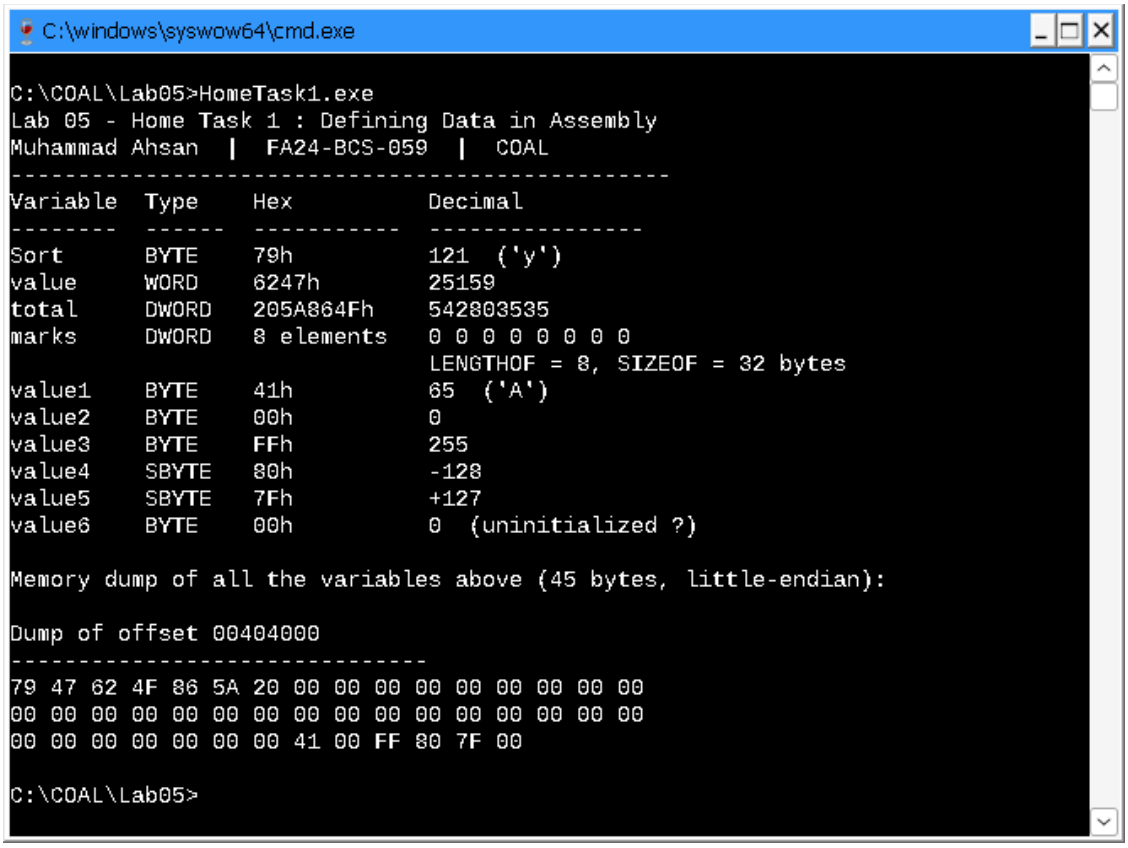


Figure 1: Output of HomeTask1.exe

Assembler listing (data section)

Offset of each variable (left) and the bytes the assembler stored for it. This listing is part of the build output (`HomeTask1.lst`).

```
00000000 79                     Sort    BYTE   'y'
00000001 4762                   value   WORD   25159
00000003 4F865A20               total   DWORD  542803535
00000007 0000000000000000       marks   DWORD  0, 0, 0, 0, 0, 0, 0, 0
00000027 41                     value1  BYTE   'A'
00000028 00                     value2  BYTE   0
00000029 FF                     value3  BYTE   255
0000002A 80                     value4  SBYTE  -128
0000002B 7F                     value5  SBYTE  +127
0000002C 00                     value6  BYTE   ?
 = 2D                           DataSize = ($ - Sort)
```

Observations

- `Sort` holds the character 'y', stored as its ASCII code **79h** (121 decimal).
- `value` = 25159 = **6247h** takes 2 bytes and is stored as `47 62`: x86 is little-endian, so the low-order byte comes first.
- `total` = 542803535 = **205A864Fh** takes 4 bytes and is stored as `4F 86 5A 20`.
- `marks` is a list of eight doublewords: LENGTHOF = 8 elements, SIZEOF = 32 bytes, all zero.
- `value2`/`value3` are the unsigned byte limits 0 (00h) and 255 (FFh); `value4`/`value5` are the signed byte limits -128 (80h in two's complement) and +127 (7Fh).
- `value6 BYTE ?` reserves one byte without giving it a value; in the `.data` segment it shows up as 00.

- All the variables together use **45 bytes** (`DataSize = ($ - Sort)` = 2Dh). MASM adds no padding, so `value` starts at offset 1 and `total` at offset 3.

Home Task 2: Menu Declared as a String

Task statement

Create the following menu by declaring a string in Assembly Language.

```
Checking Account
1) Create a new account.
2) Open an existing account
3) Credit the account
4) Debit the account
5) Exit
```

Source code: HomeTask2.asm

```
TITLE Lab 05 - Home Task 2 : Menu String              (HomeTask2.asm)

;-----------------------------------------------------------------------
; Name         : Muhammad Ahsan
; Reg. No.     : FA24-BCS-059
; Subject      : COAL
; Submitted to : Sir Taimur Shahzad
; Lab 05       : Stage v (verify) - Home Activities
;
; Home Task 2  : Create the "Checking Account" menu by declaring a
;                single string, then display it with WriteString.
;-----------------------------------------------------------------------

INCLUDE Irvine32.inc

.data
;---------- Home Task 2 : the whole menu is ONE string ----------
; A comma at the end of a line continues the same BYTE declaration,
; 0Dh,0Ah (CR, LF) moves to the next line and 0 ends the string.
menu BYTE "Checking Account",0Dh,0Ah,
          "1) Create a new account.",0Dh,0Ah,
          "2) Open an existing account",0Dh,0Ah,
          "3) Credit the account",0Dh,0Ah,
          "4) Debit the account",0Dh,0Ah,
          "5) Exit",0Dh,0Ah,0

;---------------------- strings for the output ----------------------
banner1 BYTE "Lab 05 - Home Task 2 : Menu declared as a string",0
banner2 BYTE "Muhammad Ahsan  |  FA24-BCS-059  |  COAL",0
dashes  BYTE "-----------------------------------------------",0
sSize   BYTE "SIZEOF menu = ",0
sBytes  BYTE " bytes (characters + CR/LF pairs + null terminator)",0

.code
main PROC
    ; ---- title ----
    mov   edx, OFFSET banner1
    call  WriteString
    call  Crlf
    mov   edx, OFFSET banner2
    call  WriteString
    call  Crlf
    mov   edx, OFFSET dashes
    call  WriteString
    call  Crlf
    call  Crlf

```

```
48      ; ---- display the menu (one call, one string) ----
49      mov   edx, OFFSET menu
50      call  WriteString
51
52      ; ---- size of the string in memory ----
53      mov   edx, OFFSET dashes
54      call  WriteString
55      call  Crlf
56      mov   edx, OFFSET sSize
57      call  WriteString
58      mov   eax, SIZEOF menu
59      call  WriteDec
60      mov   edx, OFFSET sBytes
61      call  WriteString
62      call  Crlf
63
64      exit
65  main ENDP
66
67  END main
```

Output

Console window after running `HomeTask2.exe`:

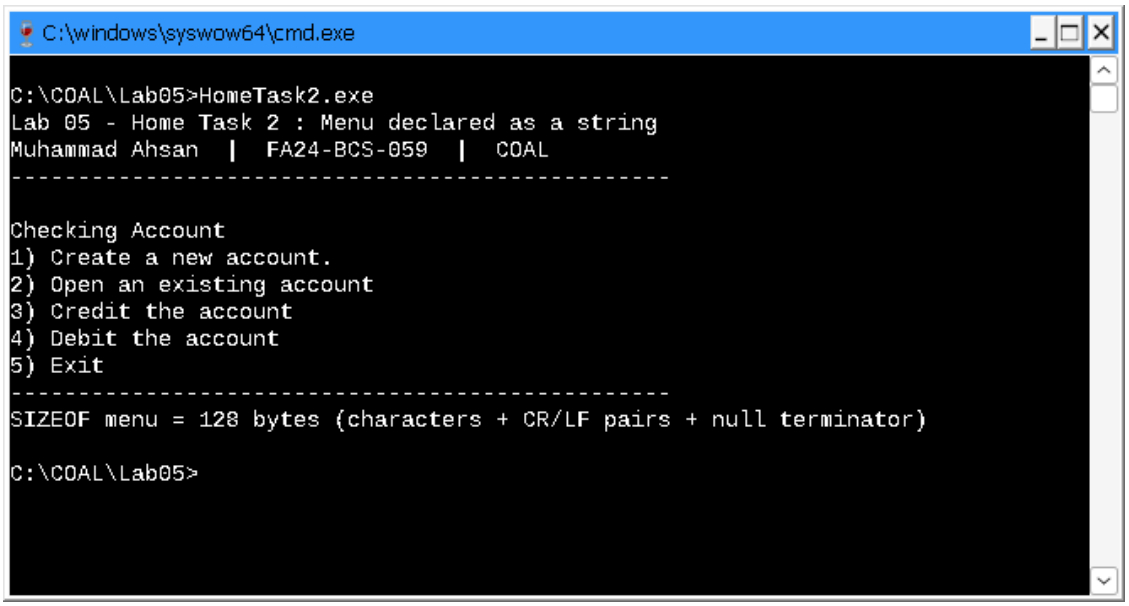


Figure 2: Output of HomeTask2.exe

Observations

- The whole menu is **one** variable, `menu`. A comma at the end of a line continues the same `BYTE` declaration onto the next line.
- `0Dh, 0Ah` (carriage return and line feed) after each line moves the cursor to the start of the next line, so a single `WriteString` call prints the complete menu.
- The final `0` is the null terminator that `WriteString` needs to know where the string ends.
- `SIZEOF menu` = **128 bytes**: 115 visible characters + 6 CR/LF pairs (12 bytes) + 1 null byte.

Home Task 3: Arrays

Task statement

Create the following arrays in Assembly Language.

```
1) Array of ten integers.
2) Array of 100 characters.
3) Arrays of 4x3 integers
4) Array of 4x3x2 integers.
```

Declarations (answer)

#	Array	Declaration	Elements	Bytes
1	intArray	DWORD 10 DUP(?)	10	40
2	charArray	BYTE 100 DUP(?)	100	100
3	array2D	DWORD 4 DUP(3 DUP(?))	4 x 3 = 12	48
4	array3D	DWORD 4 DUP(3 DUP(2 DUP(?)))	4 x 3 x 2 = 24	96

Source code: HomeTask3.asm

```
TITLE Lab 05 - Home Task 3 : Arrays                    (HomeTask3.asm)

;-----------------------------------------------------------------------
; Name         : Muhammad Ahsan
; Reg. No.     : FA24-BCS-059
; Subject      : COAL
; Submitted to : Sir Taimur Shahzad
; Lab 05       : Stage v (verify) - Home Activities
;
; Home Task 3  : Create the following arrays in assembly language
;                1) array of ten integers
;                2) array of 100 characters
;                3) array of 4 x 3 integers
;                4) array of 4 x 3 x 2 integers
;                The program shows TYPE / LENGTHOF / SIZEOF of every
;                array, fills the arrays and prints their contents.
;-----------------------------------------------------------------------

INCLUDE Irvine32.inc

.data
;------------------ Home Task 3 : array declarations ------------------
intArray  DWORD 10 DUP(?)                    ; 1) ten integers
charArray BYTE  100 DUP(?)                   ; 2) 100 characters
array2D   DWORD 4 DUP(3 DUP(?))              ; 3) 4 x 3 integers
array3D   DWORD 4 DUP(3 DUP(2 DUP(?)))       ; 4) 4 x 3 x 2 integers

;---------------------- strings for the output ----------------------
banner1 BYTE "Lab 05 - Home Task 3 : Arrays in Assembly",0
banner2 BYTE "Muhammad Ahsan  |  FA24-BCS-059  |  COAL",0
dashes  BYTE "-----------------------------------------------------------------",0
header  BYTE "   Array      Declaration                    TYPE  LENGTHOF  SIZEOF",0
hdrLine BYTE "   ---------  -----------------------------  ----  --------  ------",0

dInt    BYTE "1) intArray   DWORD 10 DUP(?)               ",0
dChar   BYTE "2) charArray  BYTE  100 DUP(?)              ",0
d2D     BYTE "3) array2D    DWORD 4 DUP(3 DUP(?))         ",0
```

```
38  d3D     BYTE "4) array3D    DWORD 4 DUP(3 DUP(2 DUP(?)))    ",0
39
40  tInt    BYTE "1) intArray : ten integers (filled with 10, 20, ... 100)",0
41  tChar   BYTE "2) charArray : 100 characters (4 rows x 25 characters)",0
42  t2D     BYTE "3) array2D : 4 rows x 3 columns (filled with 1 .. 12)",0
43  t3D     BYTE "4) array3D : 4 x 3 x 2 (filled with 1 .. 24)",0
44  n3D     BYTE "   each line is one 3 x 2 block, '|' separates its 3 rows",0
45  sRow    BYTE "   row ",0
46  sColon  BYTE " :",0
47  sBlock  BYTE "   [",0
48  sBlock2 BYTE "] :",0
49  sBar    BYTE "  |",0
50  sIndent BYTE "   ",0
51  sDump   BYTE "Memory of array2D (12 doublewords stored row after row):",0
52
53  .code
54  main PROC
55      ; ---------------- title ----------------
56      mov   edx, OFFSET banner1
57      call  WriteString
58      call  Crlf
59      mov   edx, OFFSET banner2
60      call  WriteString
61      call  Crlf
62      mov   edx, OFFSET dashes
63      call  WriteString
64      call  Crlf
65
66      ; ------- TYPE, LENGTHOF and SIZEOF of every array -------
67      mov   edx, OFFSET header
68      call  WriteString
69      call  Crlf
70      mov   edx, OFFSET hdrLine
71      call  WriteString
72      call  Crlf
73
74      mov   edx, OFFSET dInt
75      mov   eax, TYPE intArray
76      mov   ebx, LENGTHOF intArray
77      mov   ecx, SIZEOF intArray
78      call  ShowSizes
79
80      mov   edx, OFFSET dChar
81      mov   eax, TYPE charArray
82      mov   ebx, LENGTHOF charArray
83      mov   ecx, SIZEOF charArray
84      call  ShowSizes
85
86      mov   edx, OFFSET d2D
87      mov   eax, TYPE array2D
88      mov   ebx, LENGTHOF array2D
89      mov   ecx, SIZEOF array2D
90      call  ShowSizes
91
92      mov   edx, OFFSET d3D
93      mov   eax, TYPE array3D
94      mov   ebx, LENGTHOF array3D
95      mov   ecx, SIZEOF array3D
96      call  ShowSizes
97
98      ; ======== 1) intArray : store 10, 20, ... 100 and print ========
99      mov   esi, OFFSET intArray
100     mov   ecx, LENGTHOF intArray
101     mov   eax, 10
```

```
F1: mov   [esi], eax
    add   eax, 10
    add   esi, TYPE intArray
    loop  F1

    call  Crlf
    mov   edx, OFFSET tInt
    call  WriteString
    call  Crlf
    mov   edx, OFFSET sIndent
    call  WriteString
    mov   esi, OFFSET intArray
    mov   ecx, LENGTHOF intArray
P1: mov   eax, [esi]
    push  ecx
    mov   ecx, 5                        ; field width
    call  WriteDecRight
    pop   ecx
    add   esi, TYPE intArray
    loop  P1
    call  Crlf

    ; ===== 2) charArray : 4 rows of 'A' .. 'Y' (25 characters each) =====
    mov   esi, OFFSET charArray
    mov   ecx, 4                        ; rows
F2: push  ecx
    mov   al, 'A'
    mov   ecx, 25                       ; characters per row
F2c:mov   [esi], al
    inc   al
    inc   esi
    loop  F2c
    pop   ecx
    loop  F2

    call  Crlf
    mov   edx, OFFSET tChar
    call  WriteString
    call  Crlf
    mov   esi, OFFSET charArray
    mov   ecx, 4
P2: push  ecx
    mov   edx, OFFSET sIndent
    call  WriteString
    mov   ecx, 25
P2c:mov   al, [esi]
    call  WriteChar
    inc   esi
    loop  P2c
    call  Crlf
    pop   ecx
    loop  P2

    ; ===== 3) array2D : fill with 1 .. 12 and print as 4 x 3 =====
    mov   esi, OFFSET array2D
    mov   ecx, LENGTHOF array2D         ; 12 elements
    mov   eax, 1
F3: mov   [esi], eax
    inc   eax
    add   esi, TYPE array2D
    loop  F3

    call  Crlf
    mov   edx, OFFSET t2D
```

```
166      call  WriteString
167      call  Crlf
168      mov   esi, OFFSET array2D
169      mov   ebx, 0                        ; row number
170      mov   ecx, 4                        ; 4 rows
171  P3: push  ecx
172      mov   edx, OFFSET sRow
173      call  WriteString
174      mov   eax, ebx
175      call  WriteDec
176      mov   edx, OFFSET sColon
177      call  WriteString
178      mov   ecx, 3                        ; 3 columns
179  P3c:mov   eax, [esi]
180      push  ecx
181      mov   ecx, 5
182      call  WriteDecRight
183      pop   ecx
184      add   esi, TYPE array2D
185      loop  P3c
186      call  Crlf
187      inc   ebx
188      pop   ecx
189      loop  P3
190
191      ; ===== 4) array3D : fill with 1 .. 24 and print as 4 x (3 x 2) =====
192      mov   esi, OFFSET array3D
193      mov   ecx, LENGTHOF array3D         ; 24 elements
194      mov   eax, 1
195  F4: mov   [esi], eax
196      inc   eax
197      add   esi, TYPE array3D
198      loop  F4
199
200      call  Crlf
201      mov   edx, OFFSET t3D
202      call  WriteString
203      call  Crlf
204      mov   edx, OFFSET n3D
205      call  WriteString
206      call  Crlf
207      mov   esi, OFFSET array3D
208      mov   ebx, 0                        ; block number
209      mov   ecx, 4                        ; 4 blocks
210  P4: push  ecx
211      mov   edx, OFFSET sBlock
212      call  WriteString
213      mov   eax, ebx
214      call  WriteDec
215      mov   edx, OFFSET sBlock2
216      call  WriteString
217      mov   ecx, 3                        ; 3 rows in a block
218  P4r:push  ecx
219      mov   ecx, 2                        ; 2 columns in a row
220  P4c:mov   eax, [esi]
221      push  ecx
222      mov   ecx, 4
223      call  WriteDecRight
224      pop   ecx
225      add   esi, TYPE array3D
226      loop  P4c
227      pop   ecx
228      cmp   ecx, 1                        ; no bar after the last row
229      je    P4n
```

```
    mov   edx, OFFSET sBar
    call  WriteString
P4n:loop  P4r
    call  Crlf
    inc   ebx
    pop   ecx
    loop  P4

    ; ===== memory layout of array2D (row-major order) =====
    call  Crlf
    mov   edx, OFFSET sDump
    call  WriteString
    call  Crlf
    mov   esi, OFFSET array2D
    mov   ecx, LENGTHOF array2D
    mov   ebx, TYPE array2D
    call  DumpMem

    exit
main ENDP

;-----------------------------------------------------------------------
ShowSizes PROC
;
; Prints one row of the TYPE / LENGTHOF / SIZEOF table.
; Receives: EDX = OFFSET of the row label, EAX = TYPE,
;           EBX = LENGTHOF, ECX = SIZEOF
; Returns:  nothing
;-----------------------------------------------------------------------
    pushad
    call  WriteString                   ; label + declaration
    push  ecx
    mov   ecx, 4
    call  WriteDecRight                 ; TYPE
    mov   eax, ebx
    mov   ecx, 10
    call  WriteDecRight                 ; LENGTHOF
    pop   eax
    mov   ecx, 8
    call  WriteDecRight                 ; SIZEOF
    call  Crlf
    popad
    ret
ShowSizes ENDP

;-----------------------------------------------------------------------
WriteDecRight PROC
;
; Writes the unsigned value in EAX right-aligned in a field that is
; ECX characters wide (leading spaces, then the digits).
; Receives: EAX = value, ECX = field width
; Returns:  nothing (all registers preserved)
;-----------------------------------------------------------------------
    pushad
    mov   edi, ecx                      ; field width
    mov   ebx, eax                      ; keep the value
    mov   esi, 10
    mov   ecx, 0                        ; digit counter
WD1:mov   edx, 0
    div   esi                           ; EAX = EAX / 10
    inc   ecx
    cmp   eax, 0
    jne   WD1
    sub   edi, ecx                      ; spaces = width - digits
```

```
294      jle   WD3
295      mov   ecx, edi
296      mov   al, ' '
297  WD2:call  WriteChar
298      loop  WD2
299  WD3:mov   eax, ebx
300      call  WriteDec
301      popad
302      ret
303  WriteDecRight ENDP
304
305  END main
```

Output

Console window after running `HomeTask3.exe`:

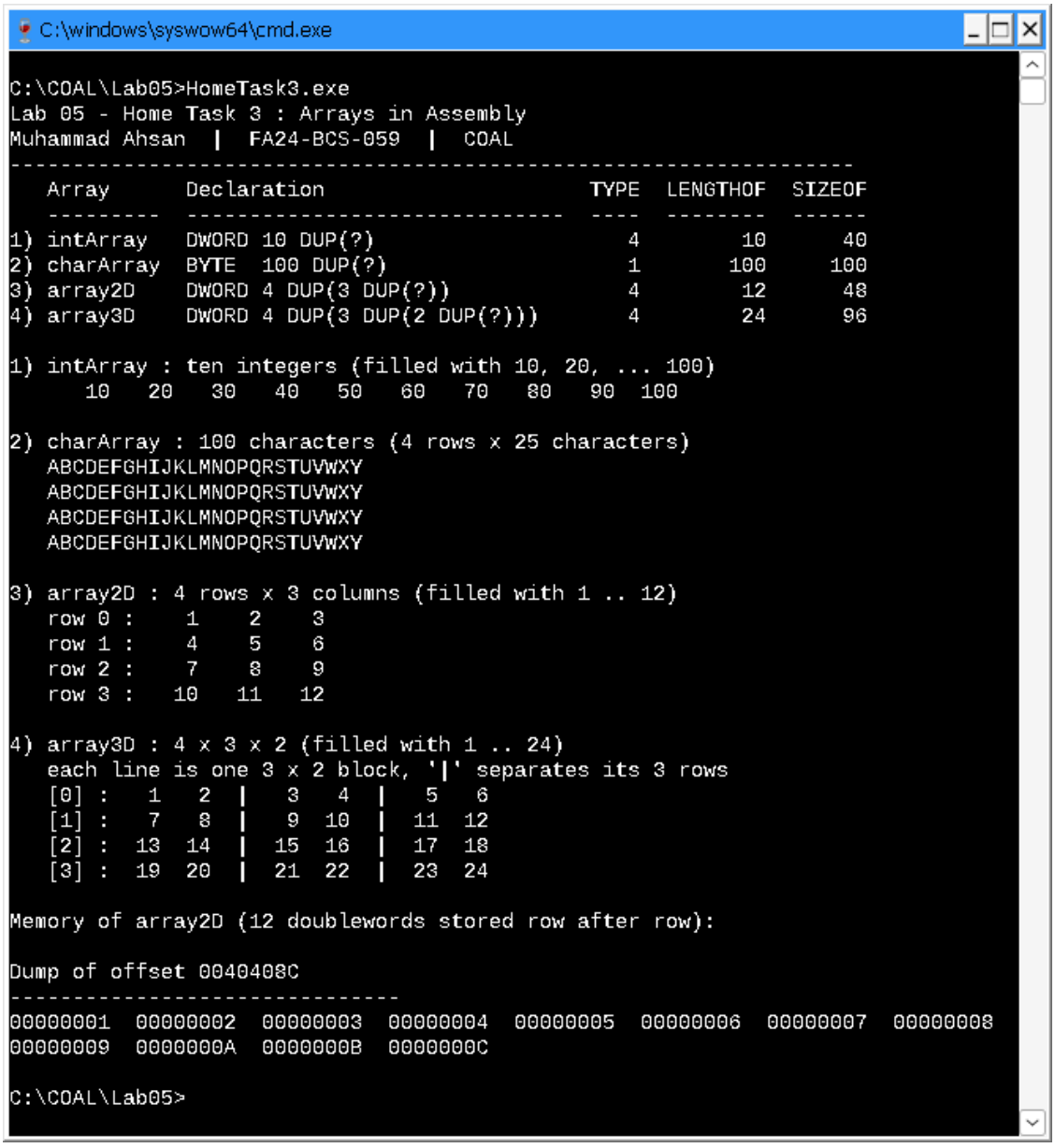


Figure 3: Output of HomeTask3.exe

Assembler listing (data section)

Offset of each variable (left) and the bytes the assembler stored for it. This listing is part of the build output (`HomeTask3.lst`).

```
00000000 0000000000000000     intArray  DWORD 10 DUP(?)
00000028 0000000000000000     charArray BYTE  100 DUP(?)
0000008C 0000000000000000     array2D   DWORD 4 DUP(3 DUP(?))
000000BC 0000000000000000     array3D   DWORD 4 DUP(3 DUP(2 DUP(?)))
```

Observations

- An integer is a 32-bit `DWORD` (TYPE = 4 bytes) and a character is a `BYTE` (TYPE = 1 byte).
- `DUP` repeats an initializer. Nesting it (`4 DUP(3 DUP(?))`) makes the 4 x 3 and 4 x 3 x 2 dimensions visible in the declaration, and `LENGTHOF` counts every element (12 and 24).
- Memory is one-dimensional, so multi-dimensional arrays are stored row by row (row-major order). Element [i][j] of the 4 x 3 array is at byte offset (i x 3 + j) x 4; element [i][j][k] of the 4 x 3 x 2 array is at ((i x 3 + j) x 2 + k) x 4. The memory dump of `array2D` (1, 2, 3, … 12) shows this order.
- The listing shows the arrays placed one after another: intArray at 0, charArray at 28h (40), array2D at 8Ch (140) and array3D at BCh (188), which matches their SIZEOF values 40, 100, 48 and 96.
- The arrays are declared with `?` (no initial values). The program fills them at run time (10 … 100, 'A' … 'Y', 1 … 12 and 1 … 24) only so that their contents can be displayed.

Home Task 4: C++ Declarations in Assembly

Task statement

Implement each of the following declarations in assembly language:

```
1) char initial;
2) char grade = 'B';
3) char x = 'P', y = 'Q';
4) int amount;
5) int count = 0;
6) int number = -396;
```

Declarations (answer)

#	C++ declaration	Assembly (MASM)	Size	Bytes in memory
1	`char initial;`	`initial BYTE ?`	1 byte	`00`
2	`char grade = 'B';`	`grade BYTE 'B'`	1 byte	`42`
3	`char x = 'P', y = 'Q';`	`x BYTE 'P'` `y BYTE 'Q'`	1 + 1 bytes	`50 51`
4	`int amount;`	`amount SDWORD ?`	4 bytes	`00 00 00 00`
5	`int count = 0;`	`count SDWORD 0`	4 bytes	`00 00 00 00`
6	`int number = -396;`	`number SDWORD -396`	4 bytes	`74 FE FF FF`

Source code: HomeTask4.asm

```
TITLE Lab 05 - Home Task 4 : C++ Declarations in MASM  (HomeTask4.asm)

;-----------------------------------------------------------------------
; Name         : Muhammad Ahsan
; Reg. No.     : FA24-BCS-059
; Subject      : COAL
; Submitted to : Sir Taimur Shahzad
; Lab 05       : Stage v (verify) - Home Activities
;
; Home Task 4  : Implement the given C/C++ declarations in assembly
;                language. A char is 1 byte (BYTE) and an int is a
;                32-bit signed integer (SDWORD).
;-----------------------------------------------------------------------

INCLUDE Irvine32.inc

.data
;----------------- Home Task 4 : the declarations -----------------
initial BYTE   ?            ; 1) char initial;
grade   BYTE   'B'          ; 2) char grade = 'B';
x       BYTE   'P'          ; 3) char x = 'P', y = 'Q';
y       BYTE   'Q'          ;    (two separate char variables)
amount  SDWORD ?            ; 4) int amount;
count   SDWORD 0            ; 5) int count = 0;
number  SDWORD -396         ; 6) int number = -396;

DataSize = ($ - initial)    ; bytes used by the declarations above

;---------------------- strings for the output ----------------------
banner1 BYTE "Lab 05 - Home Task 4 : C++ declarations in Assembly",0
banner2 BYTE "Muhammad Ahsan  |  FA24-BCS-059  |  COAL",0
```

```
dashes  BYTE "--------------------------------------------------------------------------------",0
header  BYTE "   C++ declaration         Assembly             Hex        Value",0
hdrLine BYTE "   ----------------------  -------------------  ---------  -----------------",0

lInit   BYTE "1) char initial;           initial BYTE   ?     ",0
lGrade  BYTE "2) char grade = 'B';       grade   BYTE   'B'   ",0
lX      BYTE "3) char x = 'P', y = 'Q';  x       BYTE   'P'   ",0
lY      BYTE "                           y       BYTE   'Q'   ",0
lAmount BYTE "4) int amount;             amount  SDWORD ?     ",0
lCount  BYTE "5) int count = 0;          count   SDWORD 0     ",0
lNumber BYTE "6) int number = -396;      number  SDWORD -396  ",0

sOpen   BYTE "'",0
sAscii  BYTE "'  (ASCII ",0
sClose  BYTE ")",0
sUninit BYTE "  (uninitialized)",0
sDump   BYTE "Memory dump of the declarations (",0
sDump2  BYTE " bytes, little-endian):",0

.code
main PROC
    ; ---- title ----
    mov   edx, OFFSET banner1
    call  WriteString
    call  Crlf
    mov   edx, OFFSET banner2
    call  WriteString
    call  Crlf
    mov   edx, OFFSET dashes
    call  WriteString
    call  Crlf
    mov   edx, OFFSET header
    call  WriteString
    call  Crlf
    mov   edx, OFFSET hdrLine
    call  WriteString
    call  Crlf

    ; ---- 1) char initial; ----
    mov   edx, OFFSET lInit
    call  WriteString
    movzx eax, initial
    mov   ebx, TYPE initial
    call  ShowHex
    call  WriteDec
    mov   edx, OFFSET sUninit
    call  WriteString
    call  Crlf

    ; ---- 2) char grade = 'B'; ----
    mov   edx, OFFSET lGrade
    call  WriteString
    movzx eax, grade
    mov   ebx, TYPE grade
    call  ShowHex
    call  ShowChar
    call  Crlf

    ; ---- 3) char x = 'P', y = 'Q'; ----
    mov   edx, OFFSET lX
    call  WriteString
    movzx eax, x
    mov   ebx, TYPE x
    call  ShowHex
```

```
    call  ShowChar
    call  Crlf

    mov   edx, OFFSET lY
    call  WriteString
    movzx eax, y
    mov   ebx, TYPE y
    call  ShowHex
    call  ShowChar
    call  Crlf

    ; ---- 4) int amount; ----
    mov   edx, OFFSET lAmount
    call  WriteString
    mov   eax, amount
    mov   ebx, TYPE amount
    call  ShowHex
    call  WriteInt
    mov   edx, OFFSET sUninit
    call  WriteString
    call  Crlf

    ; ---- 5) int count = 0; ----
    mov   edx, OFFSET lCount
    call  WriteString
    mov   eax, count
    mov   ebx, TYPE count
    call  ShowHex
    call  WriteInt
    call  Crlf

    ; ---- 6) int number = -396; ----
    mov   edx, OFFSET lNumber
    call  WriteString
    mov   eax, number
    mov   ebx, TYPE number
    call  ShowHex
    call  WriteInt                    ; -396
    call  Crlf

    ; ---- memory dump of all the declarations ----
    call  Crlf
    mov   edx, OFFSET sDump
    call  WriteString
    mov   eax, DataSize
    call  WriteDec
    mov   edx, OFFSET sDump2
    call  WriteString
    call  Crlf
    mov   esi, OFFSET initial         ; starting address
    mov   ecx, DataSize               ; number of units
    mov   ebx, TYPE BYTE              ; unit size = 1 byte
    call  DumpMem

    exit
main ENDP

;-----------------------------------------------------------------------
ShowHex PROC
;
; Writes EAX in hexadecimal using EBX bytes (1, 2 or 4), adds the
; suffix 'h' and pads with spaces so that the next column always
; starts at the same position.
; Receives: EAX = value, EBX = size of the value in bytes
```

```
; Returns:  nothing (EAX, EBX and ECX are preserved)
;-----------------------------------------------------------------
    push  eax
    push  ecx
    call  WriteHexB                    ; 2, 4 or 8 hex digits
    mov   al, 'h'
    call  WriteChar
    mov   ecx, 10                      ; spaces = 10 - 2 * size
    sub   ecx, ebx
    sub   ecx, ebx
    mov   al, ' '
SH1:call  WriteChar
    loop  SH1
    pop   ecx
    pop   eax
    ret
ShowHex ENDP

;-----------------------------------------------------------------
ShowChar PROC
;
; Writes the character in AL as  'C'  (ASCII nn)
; Receives: EAX = character code (0..255)
; Returns:  nothing
;-----------------------------------------------------------------
    push  edx
    push  eax
    mov   edx, OFFSET sOpen
    call  WriteString
    call  WriteChar                    ; the character itself
    mov   edx, OFFSET sAscii
    call  WriteString
    call  WriteDec                     ; its ASCII code
    mov   edx, OFFSET sClose
    call  WriteString
    pop   eax
    pop   edx
    ret
ShowChar ENDP

END main
```

Output

Console window after running `HomeTask4.exe`:

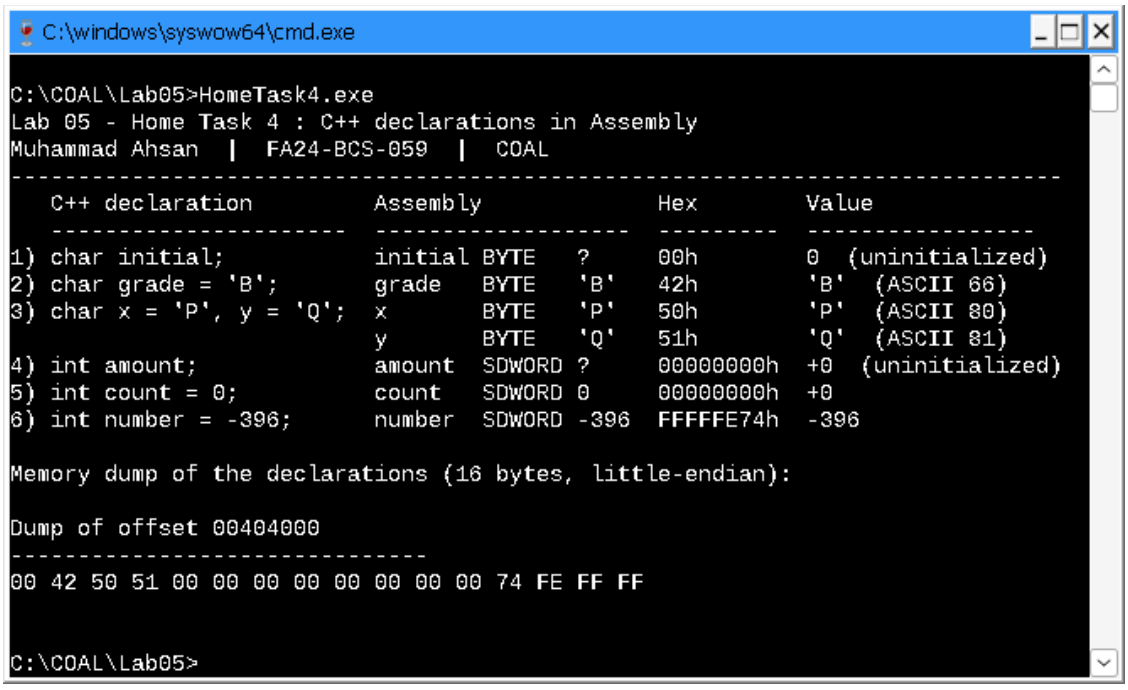


Figure 4: Output of HomeTask4.exe

Assembler listing (data section)

Offset of each variable (left) and the bytes the assembler stored for it. This listing is part of the build output (`HomeTask4.lst`).

```
00000000 00                    initial BYTE   ?
00000001 42                    grade   BYTE   'B'
00000002 50                    x       BYTE   'P'
00000003 51                    y       BYTE   'Q'
00000004 00000000              amount  SDWORD ?
00000008 00000000              count   SDWORD 0
0000000C 74FEFFFF              number  SDWORD -396
 = 10                          DataSize = ($ - initial)
```

Observations

- A C++ `char` is one byte, so it becomes `BYTE`. A C++ `int` is a 32-bit signed integer in Visual C++, so it becomes `SDWORD`.
- A declaration with no initial value (`char initial;`, `int amount;`) uses `?` in MASM. It reserves the space without a value, and in the `.data` segment it shows up as 0.
- `char x = 'P', y = 'Q';` declares two variables, so it needs two labels: `x` (50h) and `y` (51h).
- -396 in 32-bit two's complement is **FFFFFE74h**, stored little-endian as `74 FE FF FF`; `WriteInt` prints it back as -396.
- All the declarations together use 16 bytes (`DataSize` = 10h).